```cpp
#include "FibonacciSequenceIterator.h"

// Constructor - Initializes the iterator by setting fSequence to the provided FibonacciSequence pointer and fIndex to the specified starting index.
FibonacciSequenceIterator::FibonacciSequenceIterator(FibonacciSequence* aSequence, uint64_t aStart) noexcept
    : fSequence(aSequence), fIndex(aStart)
{
}

// Operator* - Returns a constant reference to the current Fibonacci number by dereferencing the underlying FibonacciSequence.
const uint64_t& FibonacciSequenceIterator::operator*() const noexcept {
    return **fSequence;
}

// Prefix Increment operator - Advances the underlying FibonacciSequence and increments the iterator index.
FibonacciSequenceIterator& FibonacciSequenceIterator::operator++() noexcept {
    ++(*fSequence);                         // Advance the Fibonacci sequence
    ++fIndex;                               // Increment the index
    return *this;                           // Return the updated iterator
}

// Postfix Increment operator - Saves the current iterator state, advances the iterator using the prefix operator, and returns the saved state.
FibonacciSequenceIterator FibonacciSequenceIterator::operator++(int) noexcept {
    FibonacciSequenceIterator temp = *this;  // Save current iterator state
    ++(*this);                               // Advance to the next state
    return temp;                             // Return the old state
}

// Equality operator - Compares the iterator by checking if both fSequence pointers and fIndex values are equal.
bool FibonacciSequenceIterator::operator==(const FibonacciSequenceIterator& aOther) const noexcept {
    return (fSequence == aOther.fSequence) && (fIndex == aOther.fIndex);
}

// begin - Resets the underlying FibonacciSequence to its initial state and returns a new iterator with index 0.
FibonacciSequenceIterator FibonacciSequenceIterator::begin() const noexcept {
    fSequence->begin();
    return FibonacciSequenceIterator(fSequence, 0);
}

// end - Returns a new iterator representing the end of the sequence by setting fIndex to MAX_FIBONACCI.
FibonacciSequenceIterator FibonacciSequenceIterator::end() const noexcept {
```

```
    return FibonacciSequenceIterator(fSequence, MAX_FIBONACCI);
}
```